

Burger Factory Project 발표 설명 노트

주제: MVC 구조, 주문 흐름, 기계 흐름, struct/enum 관계, 추가 기능, SOLID 원칙 설명

1. 전체 MVC 구조

이 프로젝트는 MVC 구조로 나누어져 있다. **Model**은 실제 데이터와 생산 로직을 담당하고, **View**는 ImGui 화면 출력과 버튼 표시만 담당하며, **Controller**는 View에서 발생한 사용자 입력을 Model 명령으로 전달한다.

```
main.cpp → BurgerFactoryModel 생성 → FactoryController 생성 → FactoryView 생성 → 매 프레임 controller.update(dt) → view.render()
```

객체 관계

- `main.cpp` 는 `BurgerFactoryModel`, `FactoryController`, `FactoryView` 를 생성하고 사용한다. 그래서 dependency 관계이다.
- `FactoryView` 는 `IFactoryViewData&` 와 `FactoryController&` 를 참조로 받는다. 직접 소유하지 않으므로 composition이 아니라 dependency이다.
- `FactoryController` 는 `IFactoryActions&` 에 의존한다. Model 전체를 알지 않고 명령 인터페이스만 안다.
- `BurgerFactoryModel` 은 `IFactoryActions`, `IFactoryViewData` 를 public inheritance로 상속한다.

발표 문장: “View는 UI만 담당하고, Controller는 버튼 입력을 Model 명령으로 전달합니다. Model은 주문, 재고, 돈, 공정 상태, 기계 상태 같은 실제 로직을 관리합니다.”

2. 주문 넣기 흐름

사용자가 버거 주문 버튼을 누르면 View가 직접 주문 데이터를 수정하지 않고 Controller를 호출한다. Controller는 `IFactoryActions` 인터페이스를 통해 Model의 `addOrder()` 를 호출한다.

```
FactoryView 버튼 → FactoryController::onNewOrder(type) →  
IFactoryActions::addOrder(type) → BurgerFactoryModel::addOrder(type) →  
OrderManager::addOrder(type) → 필요하면 enterStep(PreparingIngredients)
```

실제 코드 흐름

```
if (ImGui::Button("Classic $100", ...))  
    controller.onNewOrder(BurgerType::CLASSIC);
```

```
void FactoryController::onNewOrder(BurgerType type)  
{  
    model.addOrder(type);  
}
```

```
void BurgerFactoryModel::addOrder(BurgerType type)
{
    if (gameOver) return;
    orderManager.addOrder(type);
    statusMessage = "";
    if (currentStep == ProcessStep::Idle)
        enterStep(ProcessStep::PreparingIngredients);
}
```

BurgerFactoryModel 은 OrderManager 를 멤버 변수로 직접 가지고 있으므로 composition 관계이다. 실제 주문 저장은 OrderManager::addOrder() 가 담당한다.

3. Start 버튼 이후 기계 흐름

주문이 들어간 뒤 Start 버튼을 누르면 현재 공정 단계가 Controller로 전달되고, Model은 현재 단계와 기계 상태를 검사한 다음 실제 기계를 시작한다.

```
FactoryView Start 버튼 → FactoryController::onStartMachine(cur) →
BurgerFactoryModel::startProcess(step) → consumeIngredients() →
ProductionLine::startMachine(step) → Machine::setState(Running)
```

재료 소비

consumeIngredients() 는 BurgerFactoryModel 의 private 함수이다. 현재 주문의 버거 타입을 OrderManager 에서 가져오고, getRecipe() 로 레시피를 확인한 뒤, InventoryManager 로 재료가 충분한지 검사하고 실제 재고를 차감한다.

```
bool BurgerFactoryModel::consumeIngredients()
{
    BurgerRecipe recipe = getRecipe(orderManager.getCurrentOrder().type);
    for (const auto& [ingredient, amount] : recipe.ingredients)
        if (!inventoryManager.hasEnough(ingredient, amount)) return false;

    preparedIngredients.clear();
    for (const auto& [ingredient, amount] : recipe.ingredients)
    {
        inventoryManager.use(ingredient, amount);
        preparedIngredients[ingredient] = amount;
    }
    return true;
}
```

기계가 시작되면 productionTimerRunning 이 true가 되고, 이후 매 프레임 생산 시간이 증가한다.

4. Machine 상속과 다형성

Machine 은 추상 부모 클래스이고, 실제 기계들은 Machine 을 상속한다.

```
PrepMachine → Machine GrillMachine → Machine SauceMachine → Machine
AssemblyMachine → Machine QualityChecker → Machine PackingMachine → Machine
```

`ProductionLine` 은 `std::vector<std::unique_ptr<Machine>>` 으로 기계들을 소유한다. 따라서 `ProductionLine` ♦ `Machine` 은 composition 관계이다.

```
void ProductionLine::update(float dt)
{
    for (auto& m : machines) m->update(dt);
}
```

여기서 `m` 은 `Machine` 타입이지만 실제 객체가 `GrillMachine` 이면 `GrillMachine::update()` 가 실행된다. 이것이 다형성이다.

발표 문장: “`ProductionLine`은 `Machine` 포인터 목록만 가지고 있지만, 실제로는 각 자식 기계의 `update`가 실행됩니다. 그래서 새 기계를 추가할 때 `Machine`을 상속하고 `update`만 구현하면 됩니다.”

5. struct / enum 관계

`struct`와 `enum`은 데이터를 명확하게 표현하기 위해 사용된다. 이들은 대부분 독립 객체를 소유하는 관계가 아니라 타입을 사용하는 dependency 관계이다.

- `Order` 는 `BurgerType` 을 사용한다. 어떤 버거 주문인지 저장한다.
- `BurgerRecipe` 는 `map<IngredientType, int>` 를 가진다. 필요한 재료와 수량을 저장한다.
- `MachineConfig` 는 `BurgerType` 과 `pattyCount` 를 가진다. 기계 설정값 전달에 사용된다.
- `ProcessStep` 은 현재 공정 단계를 표현한다.
- `MachineState` 는 기계 상태를 표현한다.

`ProcessStep`: Idle → PreparingIngredients → GrillPatty → AddSauce → AssembleBurger → QualityCheck → PackBurger
`MachineState`: Idle, Running, Paused, Failed, Completed

여러 bool 변수 대신 enum 하나로 상태를 관리하면 Running, Paused, Failed가 동시에 true가 되는 문제를 막을 수 있다.

6. QualityChecker와 Packing 완료 처리

품질 검사는 `QualityChecker` 가 담당한다. 품질 검사 단계가 끝나면 `BurgerFactoryModel::nextStep()` 에서 `checkQuality()` 를 호출한다.

```
bool BurgerFactoryModel::checkQuality() const
{
    BurgerRecipe recipe = getRecipe(orderManager.getCurrentOrder().type);
    return productionLine.inspectQuality(preparedIngredients, recipe);
}
```

`ProductionLine` 은 자신이 가지고 있는 `QualityChecker` 에게 실제 검사를 맡긴다. 준비된 재료와 레시피가 정확히 일치하면 통과한다.

포장 기계가 완료되면 `PackingMachine` 은 자기 상태를 `Completed`로 바꿀 뿐, 돈 추가나 주문 완료 처리는 하지 않는다. 이 정리는 `BurgerFactoryModel::nextStep()` 의 `PackBurger` case에서 처리한다.

```
case ProcessStep::PackBurger:
{
    BurgerRecipe r = getRecipe(orderManager.getCurrentOrder().type);
    moneyManager.add(r.price);
    orderManager.completeOrder();
    productionTimerRunning = false;
    preparedIngredients.clear();
    qualityCheckPassed = false;
    if (orderManager.hasActiveOrder())
        enterStep(ProcessStep::PreparingIngredients);
    else
        enterStep(ProcessStep::Idle);
}
```

7. upgrade, refill, repair 추가 기능

Upgrade

기계 업그레이드는 cycle time을 줄이고 malfunction rate를 낮춰서 더 빠르고 안정적인 생산을 가능하게 한다. View의 upgrade 버튼은 Controller를 통해 `BurgerFactoryModel::upgradeMachine()` 을 호출한다.

Refill

재고가 부족하면 `InventoryManager` 가 재고를 보충한다. 비용은 `REFILL_COST` 로 관리된다. 재고 관리 책임은 Model Coordinator가 아니라 `InventoryManager` 에 있다.

Repair

각 Machine은 update 중 `checkMalfunction()` 을 호출한다. 고장이 발생하면 `MachineState::Failed` 가 되고, Repair 버튼을 통해 수리할 수 있다.

```
Machine::checkMalfunction() → MachineState::Failed → View Repair button →
Controller::onRepairMachine() → BurgerFactoryModel::repairMachine() →
ProductionLine::repairMachine() → Machine::reset()
```

8. SOLID 원칙 연결

S - Single Responsibility Principle

재고는 `InventoryManager`, 돈은 `MoneyManager`, 주문은 `OrderManager`, 기계 목록과 기계 실행은 `ProductionLine` 이 담당한다. 그래서 모든 기능이 하나의 클래스에 몰리지 않도록 분리했다.

O - Open/Closed Principle

새로운 기계를 추가하려면 `Machine` 을 상속하고 `update()` 를 구현하면 된다. 기존 기계 코드를 크게 수정하지 않고 확장할 수 있다.

L - Liskov Substitution Principle

`ProductionLine` 은 `Machine` 포인터로 모든 기계를 다룬다. 자식 기계들은 부모 `Machine` 처럼 사용할 수 있으므로 LSP에 맞는다.

I - Interface Segregation Principle

View는 `IFactoryViewData` 를 통해 조회 함수만 알고, Controller는 `IFactoryActions` 를 통해 명령 함수만 안다. 그래서 클라이언트가 사용하지 않는 함수에 의존하지 않는다.

D - Dependency Inversion Principle

View와 Controller는 concrete class인 `BurgerFactoryModel` 에 직접 의존하지 않고, 인터페이스에 의존한다. 이 구조는 Model 내부 구현이 바뀌어도 View와 Controller의 영향을 줄여준다.